Can Frozen Text and Code Embeddings Screen Coding-Agent Failures?

Team G04, Data Mining (Fall 2026), NYCU

Abstract

Release engineers need to identify failed coding-agent tasks before integrating their patches. We study task-end screening of 284 public SWE-agent trajectories using local, frozen MiniLM log embeddings and CodeBERT patch embeddings. A normal-reference nearest-neighbor detector reaches only 0.481 balanced accuracy in five-fold exploratory evaluation, flagging 14 of 54 successes while missing 179 of 230 failures. An inductive empirical-tail numeric control reaches 0.657 balanced accuracy, and supervised character TF-IDF reaches 0.652. Controlled comparisons show that patch-only embeddings outperform equal-weight log–patch fusion; reference contamination has small, nonmonotonic effects. We report raw confusion matrices, paired conditional uncertainty intervals, and traceable errors. No evaluated method meets our joint recall, false-alarm, and unflagged-success targets. More text information alone does not establish a reliable release gate, and failure prevalence of 81% limits the classical rare-anomaly interpretation.

1 Introduction

Coding agents produce commands, observations, and patches, but a completed run need not resolve its issue (Yang et al., 2024). Our stakeholder is a release engineer deciding which completed runs deserve extra inspection before integration. An alert requests review; absence of an alert does not authorize release.

Our application is **Category 4: Text / Logs**. Commands mix natural language, source code, tracebacks and repository conventions. Compressing them to counts loses content, but a general embedding may capture similarity rather than correctness. We ask: *Can local text/code representations improve failure screening over numeric workflow features under a common false-alarm calibration policy?*

Normal means officially resolved; failure means officially unresolved. Successful `django-13230` is falsely flagged by our detector, whereas failed `sympy-12454` is close to successful reference vectors and escapes. These labels do not establish deception, reward hacking or false completion claims. Our contributions are a shared-partition NLP/numeric comparison, controlled representation/channel/contamination studies, and traceable confusion/error analyses that expose the limits of the resulting detector.

2 Related Work

SWE-bench (Jimenez et al., 2024) evaluates patches on 2,294 GitHub issues from 12 Python repositories; its original best model resolves 1.96%. SWE-agent (Yang et al., 2024) introduces an agent-computer interface and reports 12.5% pass@1 on full SWE-bench and 87.7% on HumanEvalFix. These are issue-solving results, not directly comparable to screening metrics.

Sentence-BERT (Reimers and Gurevych, 2019) learns sentence representations for semantic comparison. We use the public MiniLM checkpoint with explicit chunk/token pooling, without fine-tuning. CodeBERT (Feng et al., 2020) pretrains on programming and natural languages; our pooled diff vectors are an experimental adaptation, not verified correctness embeddings. Both encoders run locally with frozen weights and no new generative LLM calls.

ECOD (Li et al., 2022) scores empirical marginal tails. We test an inductive variant with frozen reference distributions, not an exact reproduction of batch-dependent implementations. XGBoost (Chen and Guestrin, 2016) provides a supervised numeric comparison. These methods distinguish semantic similarity, normal-reference distance, marginal rarity and label-supervised discrimination; none is assumed to win on this

dataset.

3 Task Definition and Data

Source and labels. The public `20240402_sweagent_gpt4` SWE-bench Lite run contains 284 generated trajectories.[1] Sixteen instances with no generation are excluded. Official resolved membership defines $y = 0$ for 54 successes and $y = 1$ for 230 failures. URLs, SHA-256 hashes, official results and sample IDs are archived. Each sample is a completed trajectory and final patch, available before the external benchmark verdict. This is task-end screening, not prefix prediction.

Failure prevalence is 80.99%: this is *normal-reference failure screening, not naturally rare-event detection*. We retain the natural ratio rather than delete failures or reverse labels. This limits the HW2 anomaly framing. Always alert has perfect failure recall and average precision (AP) equal to prevalence, 0.810. Benchmark resolution itself remains an incomplete correctness proxy.

Inputs. We use actions, observations and submitted diffs. Thoughts/responses, official evaluator results, gold patches, labels, exit status and instance/repository IDs are excluded as features. ANSI escapes and workspace/instance identifiers are normalized; relative paths and error vocabulary remain. A field/hash audit of all runs and structured inspection of ten examples per class found no explicit evaluator-output signature; indirect cues cannot be ruled out.

R0 contains 16 workflow and eight patch statistics. R0-clean removes command count, which duplicates step count, leaving 23 columns; it applies $\log(1 + x)$ to 16 long-tail columns, preserves four binary flags, and robust-scales nonbinary features on successful fitting rows using median/IQR (SD, then one, if degenerate). Correlated token/cost features remain. Regex indicators are noisy: Python execution need not run tests, and a test string in a diff need not change a test file. Empty-channel masks are supported; the saved embedding cache has no empty patches.

4 Exploratory Feature Analysis

EDA uses only the designated outer-fold-0 training partition, with IDs in the shared manifest. Table 1 and Figure 1 describe activity and output magnitude. Long tails and overlapping classes motivate robust numeric preprocessing and comparison with text, not a claim that magnitude alone identifies failure. No held-out rows enter this EDA.

Feature	Success median	Failure median
Steps	12	24
Error observations	3	7
Received tokens	1377	2934
Patch characters	822	1372

Table 1: Raw training-only counts for designated outer fold 0.

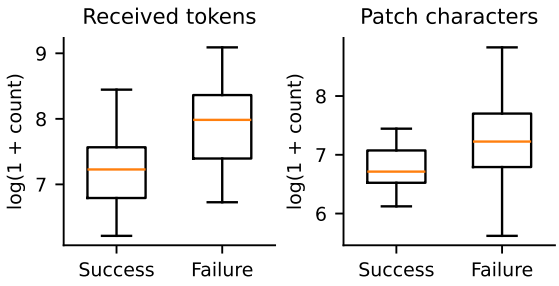


Figure 1: Training-only distributions, log-transformed for display; boxes show quartiles and outliers are omitted.

We test **H1**: dual encoders improve over numeric inputs under the same kNN family; **H2**: patch information adds value beyond logs; **H3**: failures contaminating the normal references degrade performance. These exploratory v2 hypotheses follow earlier numeric experiments, not an independent preregistration.

5 Method

Six-stage primary pipeline. (1) Normalize permitted text. (2) Encode log chunks with frozen `all-MiniLM-L6-v2`: 224 content tokens, 32 overlap, non-special token mean pooling, equal chunk averaging within steps, then equal step averaging and normalization. (3) Split patch hunks into 480-token chunks with overlap 64, encode with frozen `codebert-base`, mean-pool non-special tokens, aggregate using recorded chunk weights, and normalize. This gives 384-D log and 768-D patch vectors. Long inputs are chunked rather than silently truncated; mean pooling loses chronology.

(4) Store successful training vectors as references. For two nonempty channels,

$$d(x, r) = \frac{1}{2}\left[\frac{1-\cos(l_x, l_r)}{2} + \frac{1-\cos(p_x, p_r)}{2}\right]. \quad (1)$$

[1] https://github.com/SWE-bench/experiments/tree/main/evaluation/lite/20240402_sweagent_gpt4

Both channels empty gives channel distance zero; only one empty gives one. (5) Score by mean distance to k nearest successful references, $k \in \{1, 3, 5\}$. (6) Use validation to select k and alert threshold, then evaluate outer-test rows. Encoder inference is not fine-tuning; reference construction and calibration are separate learned components.

Tiers and controls. T1 scores error rate, 0.1 times repeated-tool streak, a final-error indicator, and 0.5 times absence of a reproduction indicator. This v2 heuristic differs from the old two-feature rule. T2 is dual encoder kNN. T3 is L2 logistic regression on normalized, equally weighted concatenated embeddings, with balanced class weights and $C \in \{.01, .1, 1, 10\}$. The character TF-IDF logistic control uses 3–5-grams, sublinear TF, $\min_df = 2$, at most 20,000 features per channel, and the same C grid. Vocabulary and IDF use only permitted fitting rows.

Numeric controls are normal-only Isolation Forest (200 trees), frozen-ECDF scoring, a numeric AE and supervised XGBoost. The ECDF variant computes smoothed fitting tail probabilities $(1 + \#\{r_j \leq x_j\})/(n+1)$ and its upper-tail counterpart, combines negative log tails with fitting skewness, and sums column contributions. Evaluation batches do not alter the fitted reference.

Secondary methods are a 23–16–4–16–23 denoising AE (10% masking, Huber numeric/BCE binary losses, equal behavior/patch group weights) and a conditional residual predictor fitted on successes: ridge ($\alpha = 10$) maps behavior to numeric patch features; logistic ($C = .1$) predicts binary outputs, with Laplace-smoothed constants for one-class targets. Unexpected outputs are not evidence of causal anomalies or intent.

6 Experimental Setup

All methods share five stratified outer folds (seed 42). Inner validation uses seed $100 + f$, giving approximately 60/20/20% fitting/validation/test per iteration, with about 32 successful fitting and ten successful validation rows. Each issue has one outer-test prediction; all its chunks stay in the same partition. Normal-reference models fit only successful training rows; supervised methods fit mixed training rows. Validation never enters weights or the reference index.

For each candidate, enumerate validation thresholds and infinities, require empirical FPR $\leq 20\%$, and maximize recall, breaking ties by balanced accuracy then higher threshold. Select the candidate with greatest resulting validation balanced accuracy, preferring smaller k/C on ties. Freeze model and threshold together, without refitting on validation. This implemented tie rule is explicit. About ten validation successes cannot certify low test FPR. Numeric random models use seeds 42/123/456; deterministic methods run once.

Primary metrics are balanced accuracy (BA), FPR, failure recall, and unflagged success fraction $TN/(TN + FN)$, with raw counts and coverage. Fold-dependent score scales make pooled raw-score AP/AUC potentially misleading; Table 2 instead gives *mean within-fold AP*. Saved pooled rankings are descriptive, not calibrated cross-fold rankings. We report fold variation separately from seed variation. Paired class-stratified bootstrap intervals use 2,000 resamples of OOF predictions; shared training folds make them conditional evidence.

These data were inspected in earlier drafts. Repartitioning does not create a fresh independent test, so v2 is exploratory. Repositories overlap. An additional leave-repository-group-out diagnostic uses fixed $k = 3$ and marks two groups unestimable; it is not the identical selected-k protocol. Original extraction did not pin encoder revisions, limiting exact future redownload reproducibility. The v2 numeric AE (24–16–8–16–24, 100 full-batch Adam epochs, lr .001) also differs from historical v1 settings.

7 Results

Main comparison. T2 reaches BA 0.481, FPR 0.259 and recall 0.222. It misses 179/230 failures and falsely flags 14/54 successes (Figure 2). T3 reaches BA 0.596, supervised TF-IDF 0.652, the ECDF variant 0.657, and secondary conditional residuals 0.675. ECDF minus T2 BA is $+0.176$, with conditional paired 95% interval $[0.114, 0.240]$; its FPR difference is -0.130 $[-0.241, -0.037]$. These intervals concern the reused data/protocol, not a new deployment population.

Representation and channel ablation. Table 4 holds the kNN family and validation policy fixed, although geometry differs: robust Euclidean numeric versus cosine text. Numeric kNN reaches BA 0.641, outperforming all tested encoder variants: H1 is unsupported. Patch-only reaches 0.594

Method	BA	FPR	Recall	TN/FP/FN/TP	Fold AP
Always alert	0.500	1.000	1.000	0/54/0/230	0.810
Never alert	0.500	0.000	0.000	54/0/230/0	0.810
T1: heuristic	0.526	0.204	0.257	43/11/171/59	0.864
T2: dual encoder kNN	0.481	0.259	0.222	40/14/179/51	0.841
T3: encoder logistic	0.596	0.352	0.543	35/19/105/125	0.905
TF-IDF logistic	0.652	0.352	0.657	35/19/79/151	0.923
Inductive ECOD variant	0.657	0.130	0.443	47/7/128/102	0.913
Isolation Forest	0.635	0.148	0.417	46/8/134/96	0.914
Conditional residual	0.675	0.185	0.535	44/10/107/123	0.908
Denoising AE	0.648	0.148	0.443	46/8/128/102	0.909
Numeric AE (v2 control)	0.651	0.185	0.487	44/10/118/112	0.904
XGBoost (v2 control)	0.646	0.278	0.570	39/15/99/131	0.908

Table 2: Seed-42 OOF operating points over 284 runs. Last column: mean within-fold AP; prevalence baseline approximately 0.810. Success/failure counts are 54/230. Secondary models do not replace T2 after observing results.

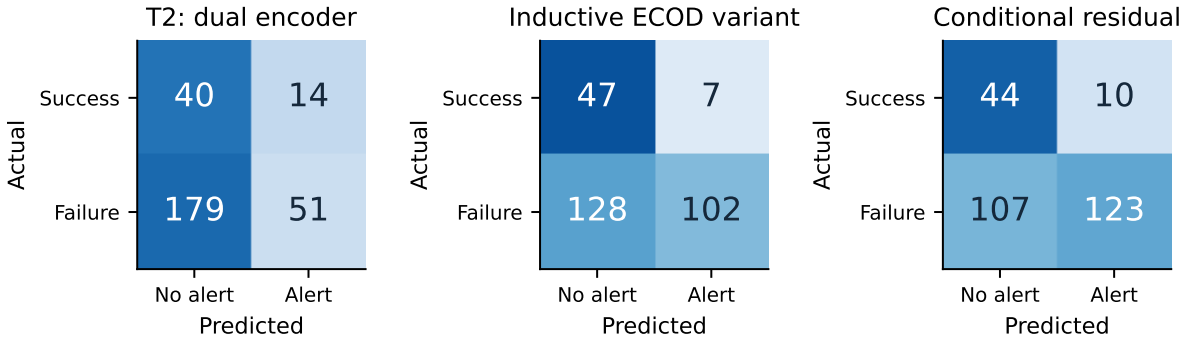


Figure 2: Raw OOF confusion counts, colored by fraction within each actual class. T2 leaves 219 runs unflagged, of which only 40 succeed.

Method	Fold BA ± SD	Seed SD
T2: dual encoder kNN	0.479 ± 0.051	–
Isolation Forest	0.632 ± 0.019	0.015
Numeric AE (v2 control)	0.647 ± 0.059	0.034
Denoising AE	0.646 ± 0.054	0.018

Table 3: Five-fold variation at seed 42; SD of pooled BA across three optimization seeds. Dash: deterministic method.

kNN input	BA	FPR	Recall
R0-clean numeric	0.641	0.167	0.448
R1 TF-IDF	0.419	0.296	0.135
R2 MiniLM logs	0.456	0.296	0.209
R3 dual encoders	0.481	0.259	0.222
CodeBERT patch only	0.594	0.185	0.374

Table 4: Representation/channel comparisons; each k uses the same validation rows.

versus log-only 0.456. Adding patches helps logs (H2 narrowly supported), but equal-weight fusion remains worse than patch-only. This implicates the tested pooling/fusion design, not all NLP approaches.

AD-specific contamination. Replace $\lfloor np \rfloor$ successful references with training failures, $p \in \{0, .05, .10\}$, keeping reference size and each clean fold's selected k fixed. Nested seeded orders choose removed successes and injected failures. IDs and actual fractions are archived; about 32 references give one/three replacements, not exactly 5/10%. Both frozen clean cutoffs and validation recalibration are tested. Zero contamination reproduces T2; an earlier mismatched $k = 3$ control was corrected before publication. Table 5 shows small, nonmonotonic changes, so H3 is unsupported here.

8 Analysis

Three errors and similarity explanations. Cases follow a fixed selection rule on T2 OOF predictions, with full IDs, SHA-256s and ex-

Requested %	Threshold	BA mean ± SD
0	clean tau	0.481 ± 0.000
0	recalibrated tau	0.481 ± 0.000
5	clean tau	0.479 ± 0.014
5	recalibrated tau	0.480 ± 0.011
10	clean tau	0.484 ± 0.007
10	recalibrated tau	0.479 ± 0.002

Table 5: Three replacement seeds, fixed-size reference sets and clean model selection.

cerpts in `case_studies.json`. Highest-score FP `django__django-13230` has $s = .11276$, $\tau = .10865$, margin $+.00411$. Its closest successful reference, `django-11133`, has log/patch distances .20290/.00655. The log dominates this pair's distance, but a small patch distance does not establish equivalent behavior. The most distant saved step is final `submit` with a diff observation; this does not establish that long debugging caused the error.

Lowest-score FN `sympy__sympy-12454` has $s = .02483$, $\tau = .07315$, margin $-.04831$. Its closest reference `sympy-13647` has distances .04656/.00311. A failed run can resemble successful vectors; similarity alone does not identify its exact semantic defect. Boundary FP `sympy__sympy-24213` has $s = .07332$, $\tau = .07315$, margin only $+.00018$, illustrating calibration instability.

Pairwise distances explain a nearest reference, while the primary score averages up to five neighbors. The saved step-removal diagnostic holds that one reference fixed instead of rescoring all kNN neighbors. It is a restricted sensitivity check, not additive attribution or causal evidence. This distinction prevents explanations from claiming more than the implementation computed.

Operational interpretation and resources. The joint exploratory targets are recall $\geq 90\%$, FPR $\leq 20\%$, unflagged success fraction $\geq 80\%$, and unflagged coverage $\geq 5\%$. No method meets them. Only 40/219 (18.3%) unflagged T2 runs succeed; even conditional residuals leave 151 unflagged with only 44 successes (29.1%). Reducing review would therefore permit many failed runs to bypass scrutiny. Neither majority-class AP nor higher BA establishes release safety.

The CPU extractor reports 894.8 seconds for all 284 runs (mean 3.14 seconds/run) and peak reported RSS 1191.5 MiB; the downstream runner reports 137.2 seconds and 939.4 MiB. These are aggregate profiler readings, not per-method training times, whole-process-tree peaks or online end-to-end latency. Missing detailed timing limits deployment comparisons. In the fixed-k repository-group diagnostic, BA spans .25–1.0 across ten estimable groups, often with very few successes; this is unstable sensitivity evidence rather than a transfer guarantee.

9 Conclusion and Limitations

Frozen text/code embeddings preserve more input content but do not outperform simple numeric controls here. Dual encoder kNN reaches BA 0.481; patch-only improves on it, while a numeric empirical-tail variant reaches 0.657. Contamination has no consistent penalty. All tested methods miss too many failures among unflagged tasks to justify automatic release.

Limitations include one old agent run, few successes, common failures, shared repositories, reused data, unpinned encoder revisions and incomplete correctness labels. Mean pooling removes order, and diff vectors omit requirements and execution semantics. Fresh task-grouped runs and task-conditioned evidence are needed. Keyword masking, task-context encoding and full kNN step-deletion rescoring were not evaluated. We claim no prefix detection, deception diagnosis, unseen-model transfer or production validation.

Contributions of Each Member

- Member 1: Pending confirmation.
- Member 2: Pending confirmation.
- Member 3: Pending confirmation.
- Member 4: Pending confirmation.
- Member 5: Pending confirmation.

AI Usage Statement. Codex assisted protocol design, implementations, local experiments, evidence checks and report/slide drafting. Earlier materials mention Claude/Gemini without confirmed versions or scope. All five member contributions and team verification remain pending confirmation at the user's request; no unconfirmed member work is attributed.

References

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. In *Findings of EMNLP*, pages 1536–1547.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.

Zheng Li, Yue Zhao, Xiyang Hu, Nicola Botta, Cezar Ionescu, and George H. Chen. 2022. Ecod: Unsupervised outlier detection using empirical cumulative distribution functions. *arXiv preprint arXiv:2201.00382*.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37.